

Advanced Python-Based Gene Sequencing Pipeline

Detailed Syntax and Workflow Explanation

github.com/chiragroy2007, IISER Tirupati

1 Introduction

This document provides a detailed breakdown of a Python function used in sequencing data preprocessing, followed by an extended explanation of a full computational gene sequencing workflow.

2 Core Function

```
def trim_read(seq, qual, threshold=20):  
    return "".join([base for base, q in zip(seq, qual) if q >=  
                    threshold])
```

3 Step-by-Step Syntax Explanation

3.1 Function Definition

- `def`: represents that we are defining a function
- `trim_read`: name of the function
- `seq`: DNA sequence (string).
- `qual`: quality scores (list of integers).
- `threshold=20`: default parameter

3.2 `zip` Function

```
zip(seq, qual)
```

Pairs corresponding elements:

```
"ATCG" + [30,10,25,18]  [( 'A',30), ( 'T',10), ...]
```

3.3 List Comprehension

```
[base for base, q in zip(seq, qual) if q >= threshold]
```

Equivalent to:

```
result = []
for base, q in zip(seq, qual):
    if q >= threshold:
        result.append(base)
```

3.4 Join Operation

```
"".join(list)
```

Concatenates characters into a string efficiently.

4 Execution Example

This section demonstrates how the `trim_read` function operates step by step using a concrete input. The goal is to understand not just the output, but how each Python construct contributes to the final result.

4.1 Input Data

We define a DNA sequence and its corresponding quality scores:

```
seq = "ATCG"
qual = [30, 10, 25, 18]
```

Interpretation:

- `seq`: A string representing nucleotide bases.
- `qual`: A list of Phred quality scores corresponding to each base.
- Each index in `qual` maps directly to the base at the same index in `seq`.

4.2 Step 1: Pairing Using `zip()`

```
list(zip(seq, qual))
```

Output:

```
[('A', 30), ('T', 10), ('C', 25), ('G', 18)]
```

Explanation:

- The `zip()` function combines two iterables element-wise.
- Each tuple contains a base and its corresponding quality score.

4.3 Step 2: Applying the Filtering Condition

We now apply the threshold condition ($q \geq 20$):

```
[(base, q) for base, q in zip(seq, qual) if q >= 20]
```

Output:

```
[('A', 30), ('C', 25)]
```

Explanation:

- Only bases with quality scores greater than or equal to 20 are retained.
- Low-quality bases such as T (10) and G (18) are removed.

4.4 Step 3: Extracting Bases Only

```
[base for base, q in zip(seq, qual) if q >= 20]
```

Output:

```
['A', 'C']
```

Explanation:

- The list comprehension now extracts only the nucleotide bases.
- Quality scores are used only for filtering and are not retained.

4.5 Step 4: Joining into Final String

```
"".join(['A', 'C'])
```

Output:

```
"AC"
```

Explanation:

- The `join()` function concatenates list elements into a string.
- The empty string `""` specifies no separator.

4.6 Final Function Execution

```
def trim_read(seq, qual, threshold=20):  
    return "".join(base for base, q in zip(seq, qual) if q >=  
                    threshold)  
  
print(trim_read(seq, qual))
```

Output:

```
AC
```

4.7 Key Observations

- The function removes individual low-quality bases rather than trimming contiguous regions.
- The mapping between sequence and quality must remain aligned.
- The use of a generator expression improves memory efficiency compared to a list.

5 Advanced Optimization

Use generator expression:

```
return "".join(base for base, q in zip(seq, qual) if q >= threshold)
```

6 Full Sequencing Workflow (Computational)

This section describes the complete computational pipeline used in gene sequencing analysis. Each step transforms raw sequencing data into biologically meaningful insights.

6.1 1. FASTQ Parsing

FASTQ files store sequencing reads along with their quality scores. Each record consists of:

- Identifier line
- Sequence line
- Separator (+)
- Quality scores (ASCII encoded)

Objective: Extract sequence and quality information into Python objects.

```
from Bio import SeqIO

for record in SeqIO.parse("reads.fastq", "fastq"):
    print(record.id)
    print(record.seq)
    print(record.letter_annotations["phred_quality"])
```

Explanation:

- `SeqIO.parse()` returns an iterator over records.
- Each record is a structured object.
- `record.seq` nucleotide sequence.
- `record.letter_annotations["phred_quality"]` quality scores.

6.2 2. Quality Controk

Sequencing errors are quantified using Phred scores:

$$Q = -10 \log_{10}(P_{error}) \quad (1)$$

Interpretation:

- Q20 1% error
- Q30 0.1% error

Stepwise Filtering:

```
def trim_read(seq, qual, threshold=20):  
    return "".join(base for base, q in zip(seq, qual) if q >=  
        threshold)  
  
seq = "ATCG"  
qual = [30, 10, 25, 18]  
  
print(trim_read(seq, qual))
```

Output:

AC

Explanation: Low-quality bases are removed before alignment.

6.3 3. Alignment

Alignment maps reads to a reference genome.

Naive Algorithm:

```
def naive_align(read, ref):  
    positions = []  
    for i in range(len(ref) - len(read) + 1):  
        if ref[i:i+len(read)] == read:  
            positions.append(i)  
    return positions  
  
print(naive_align("AT", "GATCAT"))
```

Output:

[1, 4]

Explanation:

- Sliding window comparison
- Time complexity: $O(nm)$
- Not scalable replaced by BWT/FM-index in real tools

6.4 4. BAM Processing

Aligned reads are stored in BAM format (binary compressed SAM).

Objective: Access alignment information efficiently.

```
import pysam

bam = pysam.AlignmentFile("aligned.bam", "rb")

for read in bam:
    print(read.reference_start, read.query_sequence)
```

Explanation:

- "rb" read binary mode
- reference_start genomic position
- Efficient for large datasets

6.5 5. Variant Calling)

Detect differences between reads and reference genome.

Pileup Mechanism: Stacks reads at each genomic position.

```
for pileupcolumn in bam.pileup():
    print(pileupcolumn.pos, pileupcolumn.n)
```

Output Example:

```
10023 35
10024 40
```

Explanation:

- pos genomic coordinate
- n read depth
- Used to infer SNPs/indels

6.6 6. Variant Filtering

Raw variants include noise and sequencing artifacts.

Filtering Criteria:

- Quality score
- Read depth
- Allele frequency

```
quality = 35
depth = 20

if quality > 30 and depth > 10:
    keep = True
else:
    keep = False

print(keep)
```

Output:

True

Explanation: Only high-confidence variants are retained.

6.7 7. Annotation

Maps variants to biological meaning.

Objective: Determine:

- Gene location
- Amino acid change
- Functional impact

```
import vcf

vcf_reader = vcf.Reader(open('variants.vcf'))

for record in vcf_reader:
    print(record.CHROM, record.POS, record.INFO)
```

Explanation:

- VCF = Variant Call Format
- Stores genomic variation data

6.8 8. Assembly (De Bruijn Graph)

Used when no reference genome is available.

Concept:

- Break reads into k-mers
- Construct graph of overlaps

```

from collections import defaultdict

def build_graph(reads, k):
    graph = defaultdict(list)
    for read in reads:
        for i in range(len(read) - k + 1):
            prefix = read[i:i+k-1]
            suffix = read[i+1:i+k]
            graph[prefix].append(suffix)
    return graph

reads = ["ATCG", "TCGA"]
print(build_graph(reads, 3))

```

Output (example):

```
{'AT': ['TC'], 'TC': ['CG', 'CG'], 'CG': ['GA']}
```

Explanation: Graph traversal reconstructs genome sequence.

6.9 9. Visualization

Used to interpret sequencing results.

```

import matplotlib.pyplot as plt

coverage = [10, 20, 30, 25, 15]

plt.plot(coverage)
plt.xlabel("Position")
plt.ylabel("Coverage")
plt.show()

```

Explanation:

- Visualizes read depth distribution
- Helps detect anomalies

6.10 10. Workflow Automation

Large-scale sequencing requires automated pipelines.

Snakemake Example:

```

rule align:
    input:
        "reads.fastq"
    output:
        "aligned.bam"

```

```
shell:
    "bwa mem ref.fa {input} | samtools view -Sb - > {output}"
```

Explanation:

- Defines reproducible workflows
- Handles dependencies automatically
- Scales to large datasets

7 Conclusion

Python serves as an orchestration and analysis layer in sequencing pipelines. While computationally intensive tasks are handled by optimized tools (C/C++), Python enables:

- Data parsing
- Pipeline integration
- Custom analysis
- Visualization